

ECE 60146 HW09

Andrea Goh

March 2025

1 Generative Adversarial Nets (GAN)

1.1 Theory

In this assignment, a Discriminator-Generator based Adversarial Network is used. This type of generative model is estimated via an adversarial process, which simultaneously trains a generative model G, and a discriminative model D. The goal of the generative model is to capture the data distribution, while the goal of the discriminative model is to estimate the probability that the sample came from the training data rather than G.

1.2 The Discriminator Network

The discriminator network in this section was copied directly from Professor Kak's **AdversarialLearning.py**. This discriminator network is also known as the 4-2-1 network because the first four convolutions are carried out with a kernel size of 4, stride length of 2, and padding of 1. The first four layers of the network begin with a 4-2-1 convolution, followed by a leaky ReLU activation function, and followed by a batch normalization. The output of the first four layers of the network is passed through a final convolution with a kernel size of 4, stride length of 1, and padding of 0. A Sigmoid activation function is then applied to the output of the final convolutional layer to obtain a scalar value as the final output.

```
1 from DLStudio import *
2 from AdversarialLearning import *
3
4 ##### Start: Copied from DLStudio #####
5 class DiscriminatorDG1(nn.Module):
6     def __init__(self):
7         super(DiscriminatorDG1, self).__init__()
8         self.conv_in = nn.Conv2d( 3, 64, kernel_size=4, stride=2,
9                                     padding=1)
10        self.conv_in2 = nn.Conv2d( 64, 128, kernel_size=4, stride=2,
11                                    padding=1)
12        self.conv_in3 = nn.Conv2d( 128, 256, kernel_size=4, stride=2,
13                                    padding=1)
14        self.conv_in4 = nn.Conv2d( 256, 512, kernel_size=4, stride=2,
15                                    padding=1)
16        self.conv_in5 = nn.Conv2d( 512, 1, kernel_size=4, stride=1,
17                                    padding=0)
18        self.bn1 = nn.BatchNorm2d(128)
19        self.bn2 = nn.BatchNorm2d(256)
20        self.bn3 = nn.BatchNorm2d(512)
21        self.sig = nn.Sigmoid()
22    def forward(self, x):
23        x = torch.nn.functional.leaky_relu(self.conv_in(x), negative_slope=0.2, inplace=True)
24        x = self.bn1(self.conv_in2(x))
25        x = torch.nn.functional.leaky_relu(x, negative_slope=0.2, inplace=True)
26        x = self.bn2(self.conv_in3(x))
27        x = torch.nn.functional.leaky_relu(x, negative_slope=0.2, inplace=True)
28        x = self.bn3(self.conv_in4(x))
29        x = torch.nn.functional.leaky_relu(x, negative_slope=0.2, inplace=True)
30        x = self.conv_in5(x)
31        x = self.sig(x)
32        return x
33##### End: Copied from DLStudio #####
```

1.3 The Generator Network

The generator network in this section was copied directly from Professor Kak's **AdversarialLearning.py**. The input to the generator network is a noise vector, which is a 1x1 image with 100 channels. The input to the network is first passed through a transpose convolution with a kernel size of 4, stride length of 1, and padding size of 0. Then, the next four layers follow the same structure: batch normalization $\rightarrow$ ReLU $\rightarrow$ Transpose Convolution. The transpose convolution in these four layers follow the 4-2-1 structure, where the kernel size is 4, stride length is 2, and padding is 1. Finally, the output is then passed through a Tanh activation function to generate the final output of the network, which will be the generated fake image with three channels.

```
1 ##### Start: Copied from DLStudio #####
2 class GeneratorDG1(nn.Module):
3     def __init__(self):
4         super(GeneratorDG1, self).__init__()
5         self.latent_to_image = nn.ConvTranspose2d( 100, 512, kernel_size=4, stride=1,
6             padding=0, bias=False)
7         self.upsampler2 = nn.ConvTranspose2d( 512, 256, kernel_size=4, stride=2, padding
8             =1, bias=False)
9         self.upsampler3 = nn.ConvTranspose2d( 256, 128, kernel_size=4, stride=2, padding
10            =1, bias=False)
11         self.upsampler4 = nn.ConvTranspose2d( 128, 64, kernel_size=4, stride=2, padding
12            =1, bias=False)
13         self.upsampler5 = nn.ConvTranspose2d( 64, 3, kernel_size=4, stride=2, padding
14            =1, bias=False)
15         self.bn1 = nn.BatchNorm2d(512)
16         self.bn2 = nn.BatchNorm2d(256)
17         self.bn3 = nn.BatchNorm2d(128)
18         self.bn4 = nn.BatchNorm2d(64)
19         self.tanh = nn.Tanh()
20     def forward(self, x):
21         x = self.latent_to_image(x)
22         x = torch.nn.functional.relu(self.bn1(x))
23         x = self.upsampler2(x)
24         x = torch.nn.functional.relu(self.bn2(x))
25         x = self.upsampler3(x)
26         x = torch.nn.functional.relu(self.bn3(x))
27         x = self.upsampler4(x)
28         x = torch.nn.functional.relu(self.bn4(x))
29         x = self.upsampler5(x)
30         x = self.tanh(x)
31         return x
32 ##### End: Copied from DLStudio #####
```

1.4 Training

The training code in this section was copied directly from Professor Kak's **AdversarialLearning.py**.

```
1 ##### Start: Copied from DLStudio #####
2 def run_gan_code(self, dlstudio, adversarial, discriminator, generator, results_dir):
3     dir_name_for_results = results_dir
4     if os.path.exists(dir_name_for_results):
5         files = glob.glob(dir_name_for_results + "/*")
6         for file in files:
7             if os.path.isfile(file):
8                 os.remove(file)
9             else:
10                 files = glob.glob(file + "/*")
11                 list(map(lambda x: os.remove(x), files))
12     else:
13         os.mkdir(dir_name_for_results)
14     nz = 100
15     netD = discriminator.to(self.device)
16     netG = generator.to(self.device)
17     netD.apply(self.weights_init)
18     netG.apply(self.weights_init)
19     fixed_noise = torch.randn(self.dlstudio.batch_size, nz, 1, 1, device=self.device)
20     real_label = 1
21     fake_label = 0
22     optimizerD = optim.Adam(netD.parameters(), lr=dlstudio.learning_rate, betas=(
23         adversarial.beta1, 0.999))
```

```

23     optimizerG = optim.Adam(netG.parameters(), lr=dlstudio.learning_rate, betas=(
24         adversarial.beta1, 0.999))
25     criterion = nn.BCELoss()
26     img_list = []
27     G_losses = []
28     D_losses = []
29     iters = 0
30     print("\n\nStarting Training Loop...\n\n")
31     start_time = time.perf_counter()
32     for epoch in range(dlstudio.epochs):
33         g_losses_per_print_cycle = []
34         d_losses_per_print_cycle = []
35         for i, data in enumerate(self.train_dataloader, 0):
36             netD.zero_grad()
37             real_images_in_batch = data[0].to(self.device)
38             b_size = real_images_in_batch.size(0)
39             label = torch.full((b_size,), real_label, dtype=torch.float, device=self.
40                 device)
41             output = netD(real_images_in_batch).view(-1)
42             lossD_for_reals = criterion(output, label)
43             lossD_for_reals.backward()
44             noise = torch.randn(b_size, nz, 1, 1, device=self.device)
45             fakes = netG(noise)
46             label.fill_(fake_label)
47             output = netD(fakes.detach()).view(-1)
48             lossD_for_fakes = criterion(output, label)
49             lossD_for_fakes.backward()
50             lossD = lossD_for_reals + lossD_for_fakes
51             d_losses_per_print_cycle.append(lossD)
52             optimizerD.step()
53             netG.zero_grad()
54             label.fill_(real_label)
55             output = netD(fakes).view(-1)
56             lossG = criterion(output, label)
57             g_losses_per_print_cycle.append(lossG)
58             lossG.backward()
59             optimizerG.step()
60             if i % 100 == 99:
61                 current_time = time.perf_counter()
62                 elapsed_time = current_time - start_time
63                 mean_D_loss = torch.mean(torch.FloatTensor(d_losses_per_print_cycle))
64                 mean_G_loss = torch.mean(torch.FloatTensor(g_losses_per_print_cycle))
65                 print("[epoch=%d/%d iter=%4d elapsed_time=%5d secs] mean_D_loss
66                     =%7.4f mean_G_loss=%7.4f" %
67                     ((epoch+1), dlstudio.epochs, (i+1), elapsed_time,
68                     mean_D_loss, mean_G_loss))
69                 d_losses_per_print_cycle = []
70                 g_losses_per_print_cycle = []
71                 G_losses.append(lossG.item())
72                 D_losses.append(lossD.item())
73                 if (iters % 500 == 0) or ((epoch == dlstudio.epochs-1) and (i == len(self.
74                     train_dataloader)-1)):
75                     with torch.no_grad():
76                         fake = netG(fixed_noise).detach().cpu()
77                         img_list.append(torchvision.utils.make_grid(fake, padding=1, pad_value
78                             =1, normalize=True))
79                 iters += 1
80             plt.figure(figsize=(10,5))
81             plt.title("Generator and Discriminator Loss During Training")
82             plt.plot(G_losses, label="G")
83             plt.plot(D_losses, label="D")
84             plt.xlabel("iterations")
85             plt.ylabel("Loss")
86             plt.legend()
87             plt.savefig(dir_name_for_results + "/gen_and_disc_loss_training.png")
88             plt.show()
89             images = []
90             for imgobj in img_list:
91                 img = tvtf.to_pil_image(imgobj)
92                 images.append(img)
93             imageio.mimsave(dir_name_for_results + "/generation_animation.gif", images, fps=5)
94             real_batch = next(iter(self.train_dataloader))
95             real_batch = real_batch[0]

```

```

90     plt.figure(figsize=(15,15))
91     plt.subplot(1,2,1)
92     plt.axis("off")
93     plt.title("Real_Images")
94     plt.imshow(np.transpose(torchvision.utils.make_grid(real_batch.to(self.device),
95         padding=1, pad_value=1, normalize=True).cpu(),(1,2,0)))
96     plt.subplot(1,2,2)
97     plt.axis("off")
98     plt.title("Fake_Images")
99     plt.imshow(np.transpose(img_list[-1],(1,2,0)))
100    plt.savefig(dir_name_for_results + "/real_vs_fake_images.png")
101    plt.show()
102    ##### End: Copied from DLStudio #####
103    torch.save(netD.state_dict(), 'netD.pth')
104    torch.save(netG.state_dict(), 'netG.pth')

```

1.5 Results

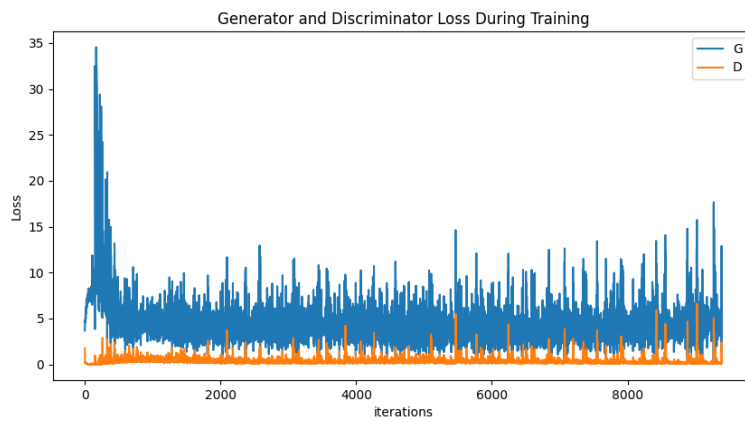


Figure 1: Adversarial Losses over Training Iterations for Generator and Discriminator



Figure 2: A 4×4 Grid of Images Generated by GAN

2 Diffusion

A pretrained diffusion model is used to generate 1024 fake images. These fake images will be used as ground truth images to refine our GAN model. The **GenerateNewImagesSamples.py** file along with the **VisualizeSamples.py** files are used to generate these images. The first step to generating these images is to run the following terminal command: `python3 GenerateNewImageSamples.py --model_path RESULTS/diffusion.pt`. Then, the **VisualizeSamples.py** file is run to generate these samples.



Figure 3: A 4×4 Grid of Images Generated by Diffusion

3 FID Score

```

1 def calculate_fid(fake_dir, device):
2     real_paths = [f'data/celeba_dataset_64x64/{file_name}' for file_name in os.listdir('
      data/celeba_dataset_64x64')]
3     fake_paths = [f'{fake_dir}/{file_name}' for file_name in os.listdir(fake_dir)]
4     dims = 2048
5     block_idx = InceptionV3.BLOCK_INDEX_BY_DIM[dims]
6     model = InceptionV3([block_idx]).to(device)
7     m1, s1 = calculate_activation_statistics(real_paths, model, device=device)
8     m2, s2 = calculate_activation_statistics(fake_paths, model, device=device)
9     fid_value = calculate_frechet_distance(m1, s1, m2, s2)
10    print(f'FID_{fake_dir}: {fid_value:.2f}')

```

Model	FID Score
GAN	117.07
Diffusion	70.64

Table 1: FID Scores

4 Finetune GAN

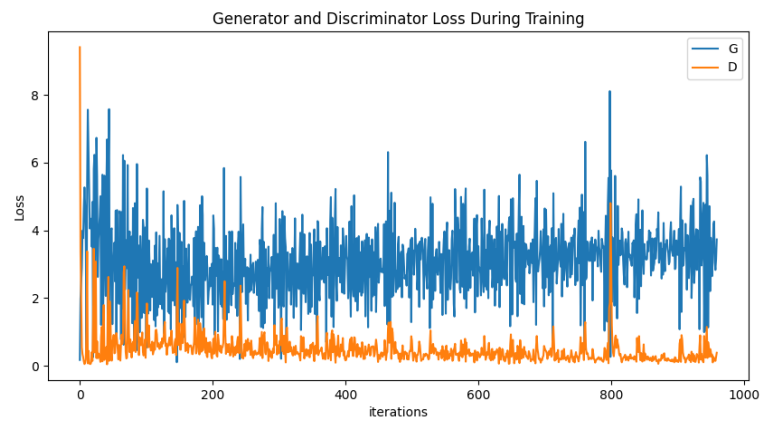


Figure 4: Adversarial Losses over Training Iterations for Generator and Discriminator



Figure 5: A 4×4 Grid of Images Generated by the Finetuned GAN

4.1 Visual Observations

The images produced by the finetuned GAN model appear significantly more realistic than those from the baseline GAN. The facial features are sharper and clearer, with fewer visual artifacts. Additionally, the skin color tones are more accurate, more closely resembling natural human skin tones, which the baseline GAN struggled to replicate.

When comparing the finetuned GAN to the diffusion model, the diffusion model still produces more lifelike faces. However, since the diffusion-generated images were used as training data to finetune the GAN, it's reasonable that the GAN's outputs don't quite match the quality of the diffusion model's generations.

5 Source Code

```
1 from DLStudio import *
2 from AdversarialLearning import *
3 import os
4 import sys
5 from PIL import Image
6 from pytorch_fid.fid_score import calculate_activation_statistics,
   calculate_frechet_distance
7 from pytorch_fid.inception import InceptionV3
8
9 ##### Discriminator-Generator DG1 #####
10 ##### Copied from DLStudio #####
11 class DiscriminatorDG1(nn.Module):
12     def __init__(self):
13         super(DiscriminatorDG1, self).__init__()
14         self.conv_in = nn.Conv2d( 3, 64, kernel_size=4, stride=2,
15                                   padding=1)
16         self.conv_in2 = nn.Conv2d( 64, 128, kernel_size=4, stride=2,
17                                   padding=1)
18         self.conv_in3 = nn.Conv2d( 128, 256, kernel_size=4, stride=2,
19                                   padding=1)
20         self.conv_in4 = nn.Conv2d( 256, 512, kernel_size=4, stride=2,
21                                   padding=1)
22         self.conv_in5 = nn.Conv2d( 512, 1, kernel_size=4, stride=1,
23                                   padding=0)
24         self.bn1 = nn.BatchNorm2d(128)
25         self.bn2 = nn.BatchNorm2d(256)
26         self.bn3 = nn.BatchNorm2d(512)
27         self.sig = nn.Sigmoid()
28     def forward(self, x):
29         x = torch.nn.functional.leaky_relu(self.conv_in(x), negative_slope=0.2, inplace=True)
30         x = self.bn1(self.conv_in2(x))
31         x = torch.nn.functional.leaky_relu(x, negative_slope=0.2, inplace=True)
32         x = self.bn2(self.conv_in3(x))
33         x = torch.nn.functional.leaky_relu(x, negative_slope=0.2, inplace=True)
34         x = self.bn3(self.conv_in4(x))
35         x = torch.nn.functional.leaky_relu(x, negative_slope=0.2, inplace=True)
36         x = self.conv_in5(x)
37         x = self.sig(x)
38         return x
39
40 class GeneratorDG1(nn.Module):
41     def __init__(self):
42         super(GeneratorDG1, self).__init__()
43         self.latent_to_image = nn.ConvTranspose2d( 100, 512, kernel_size=4, stride=1,
44                                                    padding=0, bias=False)
45         self.upsampler2 = nn.ConvTranspose2d( 512, 256, kernel_size=4, stride=2, padding
46                                               =1, bias=False)
47         self.upsampler3 = nn.ConvTranspose2d( 256, 128, kernel_size=4, stride=2, padding
48                                               =1, bias=False)
49         self.upsampler4 = nn.ConvTranspose2d( 128, 64, kernel_size=4, stride=2, padding
50                                               =1, bias=False)
51         self.upsampler5 = nn.ConvTranspose2d( 64, 3, kernel_size=4, stride=2, padding
52                                               =1, bias=False)
53         self.bn1 = nn.BatchNorm2d(512)
54         self.bn2 = nn.BatchNorm2d(256)
```

```

45         self.bn3 = nn.BatchNorm2d(128)
46         self.bn4 = nn.BatchNorm2d(64)
47         self.tanh = nn.Tanh()
48     def forward(self, x):
49         x = self.latent_to_image(x)
50         x = torch.nn.functional.relu(self.bn1(x))
51         x = self.upsampler2(x)
52         x = torch.nn.functional.relu(self.bn2(x))
53         x = self.upsampler3(x)
54         x = torch.nn.functional.relu(self.bn3(x))
55         x = self.upsampler4(x)
56         x = torch.nn.functional.relu(self.bn4(x))
57         x = self.upsampler5(x)
58         x = self.tanh(x)
59         return x
60     ##### DG1 Definition ENDS #####
61
62     def run_gan_code(self, dlstudio, adversarial, discriminator, generator, results_dir,
63                     finetune = False):
64         dir_name_for_results = results_dir
65         if os.path.exists(dir_name_for_results):
66             files = glob.glob(dir_name_for_results + "/*")
67             for file in files:
68                 if os.path.isfile(file):
69                     os.remove(file)
70             else:
71                 files = glob.glob(file + "/*")
72                 list(map(lambda x: os.remove(x), files))
73         else:
74             os.mkdir(dir_name_for_results)
75         nz = 100
76         netD = discriminator.to(self.device)
77         netG = generator.to(self.device)
78         if finetune:
79             netD.load_state_dict(torch.load('netD.pth'))
80             netG.load_state_dict(torch.load('netG.pth'))
81             dataset = torchvision.datasets.ImageFolder(root='Diffusion/',
82                                                         transform = tvn.Compose([
83                                                         tvn.Resize(self.dlstudio.image_size),
84                                                         tvn.CenterCrop(self.dlstudio.image_size),
85                                                         tvn.ToTensor(),
86                                                         tvn.Normalize((0.5, 0.5, 0.5), (0.5, 0.5,
87                                                         0.5)),
88                                                         ]))
89         self.train_dataloader = torch.utils.data.DataLoader(dataset, batch_size=self.
90             dlstudio.batch_size, shuffle=True, num_workers=self.num_workers)
91         show_iter = 10
92         netD_name = 'netD_finetune.pth'
93         netG_name = 'netG_finetune.pth'
94     else:
95         netD.apply(self.weights_init)
96         netG.apply(self.weights_init)
97         show_iter = 100
98         netD_name = 'netD.pth'
99         netG_name = 'netG.pth'
100     fixed_noise = torch.randn(self.dlstudio.batch_size, nz, 1, 1, device=self.device)
101     real_label = 1
102     fake_label = 0
103     optimizerD = optim.Adam(netD.parameters(), lr=dlstudio.learning_rate, betas=(
104         adversarial.beta1, 0.999))
105     optimizerG = optim.Adam(netG.parameters(), lr=dlstudio.learning_rate, betas=(
106         adversarial.beta1, 0.999))
107     criterion = nn.BCELoss()
108     img_list = []
109     G_losses = []
110     D_losses = []
111     iters = 0
112     print("\n\nStarting Training Loop...\n\n")
113     start_time = time.perf_counter()
114     for epoch in range(dlstudio.epochs):
115         g_losses_per_print_cycle = []
116         d_losses_per_print_cycle = []
117         for i, data in enumerate(self.train_dataloader, 0):

```

```

113         netD.zero_grad()
114         real_images_in_batch = data[0].to(self.device)
115         b_size = real_images_in_batch.size(0)
116         label = torch.full((b_size,), real_label, dtype=torch.float, device=self.
            device)
117         output = netD(real_images_in_batch).view(-1)
118         lossD_for_reals = criterion(output, label)
119         lossD_for_reals.backward()
120         noise = torch.randn(b_size, nz, 1, 1, device=self.device)
121         fakes = netG(noise)
122         label.fill_(fake_label)
123         output = netD(fakes.detach()).view(-1)
124         lossD_for_fakes = criterion(output, label)
125         lossD_for_fakes.backward()
126         lossD = lossD_for_reals + lossD_for_fakes
127         d_losses_per_print_cycle.append(lossD)
128         optimizerD.step()
129         netG.zero_grad()
130         label.fill_(real_label)
131         output = netD(fakes).view(-1)
132         lossG = criterion(output, label)
133         g_losses_per_print_cycle.append(lossG)
134         lossG.backward()
135         optimizerG.step()
136         if i % show_iter == show_iter-1:
137             current_time = time.perf_counter()
138             elapsed_time = current_time - start_time
139             mean_D_loss = torch.mean(torch.FloatTensor(d_losses_per_print_cycle))
140             mean_G_loss = torch.mean(torch.FloatTensor(g_losses_per_print_cycle))
141             print("[epoch=%d/%d, iter=%4d, elapsed_time=%5d, secs] mean_D_loss
                =%7.4f, mean_G_loss=%7.4f" %
142                   ((epoch+1), dlstudio.epochs, (i+1), elapsed_time,
                    mean_D_loss, mean_G_loss))
143             d_losses_per_print_cycle = []
144             g_losses_per_print_cycle = []
145             G_losses.append(lossG.item())
146             D_losses.append(lossD.item())
147             if (iters % 500 == 0) or ((epoch == dlstudio.epochs-1) and (i == len(self.
                train_dataloader)-1)):
148                 with torch.no_grad():
149                     fake = netG(fixed_noise).detach().cpu()
150                     img_list.append(torchvision.utils.make_grid(fake, padding=1, pad_value
                        =1, normalize=True))
151             iters += 1
152         plt.figure(figsize=(10,5))
153         plt.title("Generator and Discriminator Loss During Training")
154         plt.plot(G_losses, label="G")
155         plt.plot(D_losses, label="D")
156         plt.xlabel("iterations")
157         plt.ylabel("Loss")
158         plt.legend()
159         plt.savefig(dir_name_for_results + "/gen_and_disc_loss_training.png")
160         plt.show()
161         images = []
162         for imgobj in img_list:
163             img = tvtf.to_pil_image(imgobj)
164             images.append(img)
165         imageio.mimsave(dir_name_for_results + "/generation_animation.gif", images, fps=5)
166         real_batch = next(iter(self.train_dataloader))
167         real_batch = real_batch[0]
168         plt.figure(figsize=(15,15))
169         plt.subplot(1,2,1)
170         plt.axis("off")
171         plt.title("Real Images")
172         plt.imshow(np.transpose(torchvision.utils.make_grid(real_batch.to(self.device),
            padding=1, pad_value=1, normalize=True).cpu(), (1,2,0)))
173         plt.subplot(1,2,2)
174         plt.axis("off")
175         plt.title("Fake Images")
176         plt.imshow(np.transpose(img_list[-1], (1,2,0)))
177         plt.savefig(dir_name_for_results + "/real_vs_fake_images.png")
178         plt.show()
179         torch.save(netD.state_dict(), f'{netD_name}.pt')

```

```

180     torch.save(netG.state_dict(), f'{netG_name}.pt')
181
182     dls = DLStudio(
183         dataroot = "data",
184         image_size = [64, 64],
185         path_saved_model = "./saved_model",
186         learning_rate = 1e-4,
187         epochs = 30,
188         batch_size = 32,
189         use_gpu = True,
190     )
191
192     adversarial = AdversarialLearning(
193         dlstudio = dls,
194         ngpu = 1,
195         latent_vector_size = 100,
196         beta1 = 0.5,
197     )
198
199     def run_training():
200         dcgan = AdversarialLearning.DataModeling( dlstudio = dls, adversarial =
            adversarial )
201
202         discriminator = DiscriminatorDG1()
203         generator = GeneratorDG1()
204
205         num_learnable_params_disc = sum(p.numel() for p in discriminator.parameters() if p.
            requires_grad)
206         print("\n\nThe number of learnable parameters in the Discriminator: %d\n" %
            num_learnable_params_disc)
207         num_learnable_params_gen = sum(p.numel() for p in generator.parameters() if p.
            requires_grad)
208         print("\n\nThe number of learnable parameters in the Generator: %d\n" %
            num_learnable_params_gen)
209         num_layers_disc = len(list(discriminator.parameters()))
210         print("\n\nThe number of layers in the 32, 100, 1, 1 discriminator: %d\n" %
            num_layers_disc)
211         num_layers_gen = len(list(generator.parameters()))
212         print("\n\nThe number of layers in the generator: %d\n\n" % num_layers_gen)
213
214         dcgan.set_dataloader()
215
216         print("\n\nHere is one batch of images from the training dataset:")
217         dcgan.show_sample_images_from_dataset(dls)
218         run_gan_code(dcgan, dls, adversarial, discriminator=discriminator, generator=
            generator, results_dir="results_DG1")
219
220     import torchvision.transforms.functional as F
221
222     def generate_img_samples(num_fakes, device, model_path, save_dir):
223         for i in range(num_fakes):
224             noise = torch.randn(1, 100, 1, 1, device=device)
225             netG = GeneratorDG1().to(device)
226             netG.load_state_dict(torch.load(model_path))
227             fakes = (netG(noise).detach().cpu())/2 + 0.5
228             img = F.to_pil_image(fakes[-1])
229             img.save(f'{save_dir}/{i}.jpg')
230
231     def calculate_fid(fake_dir, device):
232         real_paths = [f'data/celeba_dataset_64x64/{file_name}' for file_name in os.listdir('
            data/celeba_dataset_64x64')]
233         fake_paths = [f'{fake_dir}/{file_name}' for file_name in os.listdir(fake_dir)]
234         dims = 2048
235         block_idx = InceptionV3.BLOCK_INDEX_BY_DIM[dims]
236         model = InceptionV3([block_idx]).to(device)
237         m1, s1 = calculate_activation_statistics(real_paths, model, device=device)
238         m2, s2 = calculate_activation_statistics(fake_paths, model, device=device)
239         fid_value = calculate_frechet_distance(m1, s1, m2, s2)
240         print(f'FID({fake_dir}): {fid_value:.2f}')
241
242     def finetune():
243         dcgan = AdversarialLearning.DataModeling( dlstudio = dls, adversarial =
            adversarial )

```

```

244     discriminator = DiscriminatorDG1()
245     generator = GeneratorDG1()
246     run_gan_code(dcgan, dls, adversarial, discriminator=discriminator, generator=
        generator, results_dir="results_finetune", finetune=True)
247
248     device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')
249     # run_training()
250     # generate_img_samples(1024, device, 'netG.pth', 'GAN') # Generate 1024 samples from GAN
        model
251     # generate_img_samples(100, device, 'netG_finetune.pth', 'GAN Finetune')
252     # calculate_fid('Diffusion', device)
253     # calculate_fid('GAN', device)
254     # finetune()

```
